Bacha Hua Khana: Technical Overview

Oct 24, 2026 · @Prathik Joe Paul

Overview

Bacha Hua Khana ("leftover food") lets a kitchen report surplus food by speaking, in Hindi, Malayalam, Kannada or English. The app reads the details back by voice and only accepts the report once staff say it is right. It is our entry for Gnani AI's Great Indian AI Internship Challenge (deadline 10 Nov 2026), and all speech runs on Gnani's APIs.

The full product idea: a kitchen speaks a report, the app confirms it, then finds the nearest volunteer and alerts them by voice in their language. **Phase 1, the kitchen loop, is built and tested.** Volunteer matching and the dashboard are Phase 2.

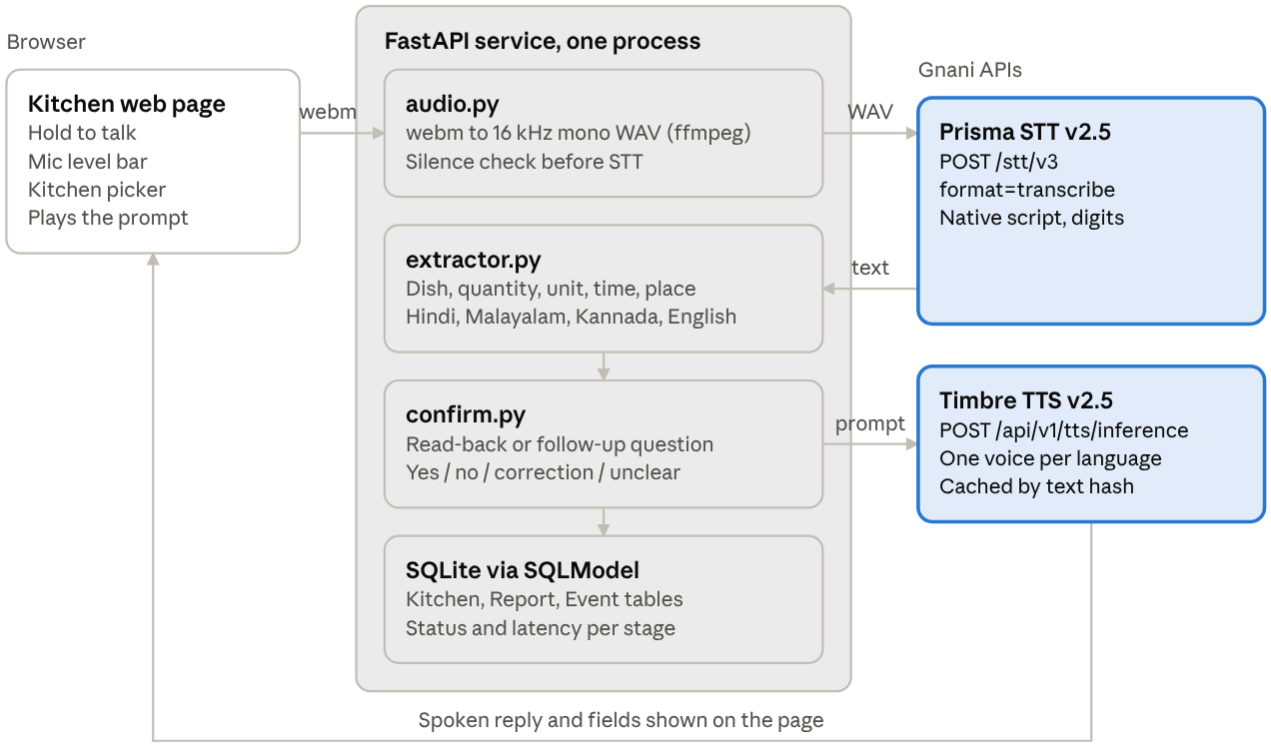
What Phase 1 does today:

- A staff member holds a button and speaks, e.g. "pachaas plate veg biryani hai, do baje bana tha, Kakkanad se".
- The app transcribes it, pulls out dish, quantity, unit, cooked time and location, and works out a pickup deadline.
- It speaks the details back in the kitchen's language and asks "Sahi hai?" ("Is that right?").
- Staff answer yes, no, or a correction ("nahi, saath plate"). Missing details trigger a spoken follow-up question.
- On a yes, the report reaches CONFIRMED. Nothing is acted on until a person has confirmed it by voice.

It is a prototype: one service, synthetic kitchens and phone numbers around Kochi, no login.

Architecture

One FastAPI process does everything: it serves the page, converts audio, calls Gnani, understands the report and stores it.



request flow · browser, one FastAPI service, two Gnani APIs

A recording goes left to right through STT, comes back as text, is turned into fields and a prompt, and returns as spoken audio. The same path runs for every reply until staff say yes.

Tech stack

Everything is one Python service plus a plain HTML page, with Gnani as the only external service.

Layer	What we use	Why
Speech-to-text	Gnani Prisma v2.5 (REST, `/stt/v3`)	Challenge requirement; 10 Indian languages; returns numbers as digits
Text-to-speech	Gnani Timbre v2.5 (REST, `/api/v1/tts/inference`)	Challenge requirement; 42 voices across Indian languages
Language	Python 3.10+ (CLAUDE.md targets 3.11)	Team familiarity, fast prototyping

Layer	What we use	Why
Web framework	FastAPI + Uvicorn	Typed endpoints, file uploads, serves the page too
Database	SQLite via SQLModel (on SQLAlchemy + Pydantic)	Zero setup; typed tables shared with the API
HTTP client	httpx	Calls to Gnani with timeouts
Audio conversion	pydub + ffmpeg	Browser records webm/opus; Prisma wants WAV, so we convert to 16 kHz mono 16-bit
Config	python-dotenv, `backend/.env`	Key and settings stay out of code and git
Understanding reports	Our own rule-based extractor (no LLM yet)	Free, fast (1-2 ms), deterministic; swappable for an LLM later
Frontend	Plain HTML, CSS, JavaScript; MediaRecorder and Web Audio APIs	No build step; works on phones
Tests	pytest + FastAPI TestClient	123 tests, all in mock mode, no credits spent
Dev tooling	Windows, PowerShell, VS Code, winget for ffmpeg	The team's machines

Not used, on purpose: no Docker, no auth, no microservices, no other speech or AI providers. Python packages are listed in `backend/requirements.txt`.

Gnani speech APIs

Both calls go through one small client, `backend/app/gnani.py`, against `https://api.vachana.ai` with the header `X-API-Key-ID` read from `backend/.env`.

Call	Endpoint	What we send	What we get
Prisma STT	`POST /stt/v3` (multipart)	`audio_file` (16 kHz mono WAV), `language_code` (e.g. `ml-IN`), `format=transcribe`	JSON with `transcript`, e.g. "50 പ്ലേറ്റ് ... 02:00 മണിക്ക്"

Call	Endpoint	What we send	What we get
Timbre TTS	`POST /api/v1/tts/inference` (JSON)	`text`, `voice`, `model: timbre-v2.5`, `language`, `speed`, `audio_config` (24 kHz, mono, 16-bit PCM, WAV)	Binary WAV

How we keep credit use low (we have 5,000):

- **TTS cache.** Each prompt is cached on disk under a hash of text, voice, language and speed. A sentence spoken twice costs one credit.
- **Mock mode.** With `MOCK_GNANI=1`, STT returns a canned or typed transcript and TTS writes a short silent WAV. The whole app and all tests run with no credits. Mock files use a `mock-` prefix so they are never served in live mode.
- **Silence check.** Before any STT call the server rejects empty uploads and clips with under 300 ms of sound, so a dead microphone costs nothing.

`format=transcribe` turns on Gnani's inverse text normalisation: spoken numbers and times come back as digits ("ಐವತ್ತು" becomes "50", "two in the afternoon" becomes "02:00"). It is set with `STT_FORMAT` and can be switched to `verbatim`.

Languages

Four languages work end to end. Each kitchen has a language, and that one setting picks the STT language, the TTS voice and the language of every spoken prompt.

Language	STT code	TTS voice	STT returns	Demo kitchen
Hindi	`hi-IN`	Nalini	Devanagari	Annapurna Mess, Kakkanad
Malayalam	`ml-IN`	Reshma	Malayalam script	Malabar Kitchen, Edappally
Kannada	`kn-IN`	Saanvi	Kannada script	Udupi Grand, Kaloor
English	`en-IN`	Kaveri	Latin; also accepts Hinglish speech	Harbour Cafe, Fort Kochi

How it is built:

- **One module per language** in `backend/app/languages/` (`hi.py`, `ml.py`, `kn.py`, `en.py`). Each holds the STT code, voice, word lists (numbers, units, time words, dishes, yes/no) and the prompt templates. Adding a language means copying one file and translating it.
- **Word lists are merged.** The extractor uses every language's words at once. Scripts don't collide, and Kerala staff mix languages ("50 plate ബിരിയാണി", "ok").
- **Glued word endings.** Malayalam and Kannada attach endings to words (മണിക്ക് = മണി + ക്ക്, "at ... o'clock"). Words in those scripts are matched by their beginning, the longest dictionary word first, and a dropped final virama is allowed for (സാമ്പാറും still reads as sambar).
- **Negation inside a word.** ശരിയല്ല and ಸರಿಯಿಲ್ಲ ("not right") count as no; look-alikes such as നല്ലത് ("good") do not.

We checked this against real Gnani output with a round trip: 10 sentences were spoken by Gnani TTS and transcribed by Gnani STT (20 credits). After two fixes (Kannada "kg" written as two words, and new place-name spellings), the extractor got every field right on 9 of 10. The tenth was an English sentence STT cut down to "01:30". Those real transcripts are now regression tests.

Understanding reports

A rule-based extractor turns the transcript into five fields, and a confirmation loop makes sure a person agrees before anything happens.

Extractor (`backend/app/extractor.py`). It splits the transcript into words and reads:

Field	Examples it understands	Required
Quantity	50, pachaas, do sau, dedh sau, അമ്പത്, ಐವತ್ತು, twenty five	Yes
Unit	plate, kg, litre, box (plus thali, dabba, പ്ലേറ്റ്, ಕೆಜಿ; "50 logon ka khana" counts as plates)	Yes
Dish	biryani, dal chawal, ചോറ് സാമ്പാറ്, ಬಿಸಿಬೇಳೆಬಾತ್, "50 plates of pasta"	Yes
Cooked time	do baje, subah 11, saadhe 12, 2 pm, 02:00, ഉച്ചയ്ക്ക് 2 മണിക്ക്	Yes
Location	Kochi area names in any script, "Kakkanad se", "from Fort Kochi"	No (the kitchen's address is known)

A few rules matter:

- A time without am/pm becomes the most recent past time, since the food is already cooked. Midnight to 5 am is ruled out.
- Ambiguous words only count as numbers next to a unit ("kar do" is not "two").
- The pickup deadline is cooked time plus `SAFE_WINDOW_HOURS` (4 by default).
- It sits behind an `Extractor` interface, so an LLM extractor can replace it without changing anything else.

Confirmation loop (`backend/app/confirm.py`). Each spoken reply is sorted into one of four kinds:

Reply kind	How it is detected	What happens
Correction	The reply contains any field ("nahi, saath plate hai")	Only those fields change; the app reads back again
No	A no-word (nahi, galat, ಇಲ್ಲ, ಇಲ್ಲ, no); no beats yes	Asks staff to say the details again
Yes	A yes-word (haan, sahi, ശരി, ಹೌದು, ok) and nothing missing	Report becomes CONFIRMED
Unclear	None of the above	Says it didn't understand and repeats the question

If a required field is missing, the app asks for it ("Kitna khana bacha hai?") instead of reading back, and a bare answer like "pachaas" fills it. Yes and no words from all four languages are accepted whatever the kitchen's language.

Data model and states

Five SQLite tables, defined with SQLModel in `backend/app/models.py`. Phase 1 uses Kitchen, Report and Event; Volunteer and Offer exist for Phase 2.

Table	Fields	Used for
Kitchen	id, name, lat, lng, phone, language	The 4 demo kitchens; `language` drives STT, TTS and prompts
Report	id, kitchen_id, transcript, dish, quantity, unit, cooked_at, location, deadline, status, created_at	One food report and where it is in the flow

Table	Fields	Used for
Event	id, report_id, type, payload_json, latency_ms, ts	Audit trail: every status change, STT/extract/TTS timing, replies, silent clips
Volunteer	id, name, lat, lng, phone, language, capacity, available	Phase 2 matching
Offer	id, report_id, volunteer_id, rank, status, sent_at, responded_at, reply_transcript	Phase 2 offers and fallbacks

A report moves through these states. Phase 1 covers the first five:

1. RECORDED: audio uploaded.
2. TRANSCRIBED: Prisma returned text.
3. EXTRACTED: fields pulled out.
4. CONFIRMING: the app has spoken a read-back or question. It stays here through any number of corrections, no's and follow-ups.
5. CONFIRMED: staff said yes to a complete report.
6. MATCHING, OFFERED, ASSIGNED, PICKED_UP: Phase 2.

Times are stored as Kochi wall-clock time (IST), so "do baje" means 2 pm in Kochi whatever the server's timezone. All data is synthetic.

HTTP API

All endpoints live in `backend/app/main.py`, served by FastAPI on one port.

Method	Path	Input	Returns
POST	`/reports/audio`	multipart `audio_file`, `kitchen_id`, optional `mock_transcript`	New report: transcript, fields, missing fields, prompt text, `audio_url`, per-stage latency
POST	`/reports/{id}/reply`	multipart `audio_file`, optional `mock_transcript`	Reply kind (yes/no/correction/unclear), updated fields, new status, next prompt and audio

Method	Path	Input	Returns
GET	`/tts/{file}`	Cached audio file name	The WAV to play; other names get 404
GET	`/kitchens`	none	Kitchens with language names and a sample sentence
GET	`/config`	none	Mock flag, safe window, supported languages
GET	`/kitchen/`	none	The kitchen web page

Errors come back as readable messages: 400 for empty or undecodable audio, 409 if a reply arrives for a report that is not CONFIRMING, 422 for silent audio, and 502 if Gnani fails. The key never appears in responses or logs.

Kitchen web page

One mobile-first page (`frontend/kitchen/` : `index.html` , `style.css` , `app.js`), served by the same FastAPI app with no build step.

- **Kitchen picker** at the top. Choosing a kitchen also chooses its language, and the choice is remembered.
- **Hold-to-talk button**, about 260 px across. Recording runs while it is held (mouse, touch, or Space/Enter) using the browser's MediaRecorder, as webm/opus where supported.
- **Live mic level bar** (Web Audio API) so staff can see the microphone is hearing them. Silent recordings are caught in the browser and never uploaded.
- **Microphone picker**, shown when there is more than one input or after a silent recording.
- **Result card** with transcript, dish, quantity, cooked time, pickup deadline and location. Missing fields are highlighted.
- **Spoken prompt** plays automatically, with a "Play again" button and the text shown.
- **Mock panel** (mock mode only): type what you would say and send it without a microphone.
- Large touch targets (48 px minimum), works at phone width, latency per stage shown at the bottom.

The microphone needs `localhost` or HTTPS, so testing on a phone needs a tunnel such as ngrok.

Reliability and observability

Every stage is timed and logged, and the common failure modes we hit in testing are handled before they cost credits.

- **Latency per stage.** `convert`, `stt`, `extract` and `tts` times (ms) go into the Event table and are shown on the page. In the round trip, Prisma STT took 0.7 to 2.0 s per clip; extraction takes 1 to 2 ms.
- **Silence detection.** pydub measures how much of the clip is louder than -45 dBFS. Under 300 ms means no speech, and STT is skipped. Real speech measured 2.7 s; the dead-mic clips we hit measured 38 to 123 ms.
- **ffmpeg on Windows.** If ffmpeg was installed after the terminal opened, the server finds it in the winget folder anyway. `FFMPEG_DIR` in `.env` overrides.
- **Recording bugs fixed.** Quick repeated taps used to send empty files; each recording now keeps its own data, and a dead mic stream is reopened.
- **Secrets.** The Gnani key lives only in `backend/.env`, which is gitignored, and is never printed.

Testing

123 pytest tests pass, all in mock mode against a throwaway database, so running them spends no credits.

File	Covers
`test_extractor.py`	13 Hindi/Hinglish/English report sentences, missing fields, follow-up answers, time rules
`test_confirm.py`	Yes/no/correction/unclear classification, correction merging, follow-up questions, Hindi read-back wording
`test_languages.py`	14 Malayalam/Kannada/English sentences, 6 real Gnani transcripts, glued words, read-back in each language, full flow per kitchen
`test_api.py`	Report to correction to yes to CONFIRMED over HTTP, latency events logged, bad audio names rejected
`test_audio.py`	Silence detection; empty and silent uploads rejected before STT in live mode

Run them from `backend/` with `python -m pytest -q`.

Setup and run

On Windows with PowerShell, from the project folder:

```
cd backend
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
copy .env.example .env          # then set GNANI_API_KEY
winget install Gyan.FFmpeg      # live mode only

# free demo, no credits
$env:MOCK_GNANI="1"; uvicorn app.main:app --reload --port 8000

# real speech
$env:MOCK_GNANI="0"; uvicorn app.main:app --reload --port 8000
```

Then open `http://localhost:8000/kitchen/`, pick a kitchen and hold the button.

Setting in `backend/.env`	Default	Meaning
`GNANI_API_KEY`	none	Gnani key from app.gnani.ai
`MOCK_GNANI`	1	1 = canned speech, no credits
`STT_FORMAT`	transcribe	transcribe = numbers as digits; verbatim = as spoken
`TTS_MODEL`	timbre-v2.5	Timbre model
`SAFE_WINDOW_HOURS`	4	Pickup deadline after cooking
`DATABASE_URL`	sqlite:///./bacha.db	Database file
`FFMPEG_DIR`	none	ffmpeg folder if not on PATH

Known limitations and next steps

Limitations today:

- No native speaker has checked the Malayalam and Kannada prompts and word lists yet.
- English STT once cut a full sentence down to "01:30" with `format=transcribe`; not yet re-tested with `verbatim`.

- STT can drop words (Kannada "ಅನ್ನ" was lost in one sample). The spoken read-back is there to catch this.
- The rule-based extractor only knows the words in its lists; an unusual dish or phrasing needs a follow-up question.
- A rejected first recording leaves an empty RECORDED report in the database.
- Single SQLite file, no auth: fine for a prototype, not for production.

Next steps:

- [ ] Phase 2: seed volunteers, rank by distance and capacity, offer by voice in their language, fall back to the next volunteer on "no" or timeout (`OFFER_TIMEOUT_SECONDS`).
- [ ] Live dashboard over WebSockets showing reports, offers and stage latencies.
- [ ] Native-speaker review of `ml.py` and `kn.py` .
- [ ] Re-test English with `verbatim` (1 credit) and pick the STT format per language.
- [ ] Optional LLM extractor behind the existing interface, kept within the Gnani-only speech rule.